

نظام تسجيل الحضور الذكي - Access Point شرح نظام

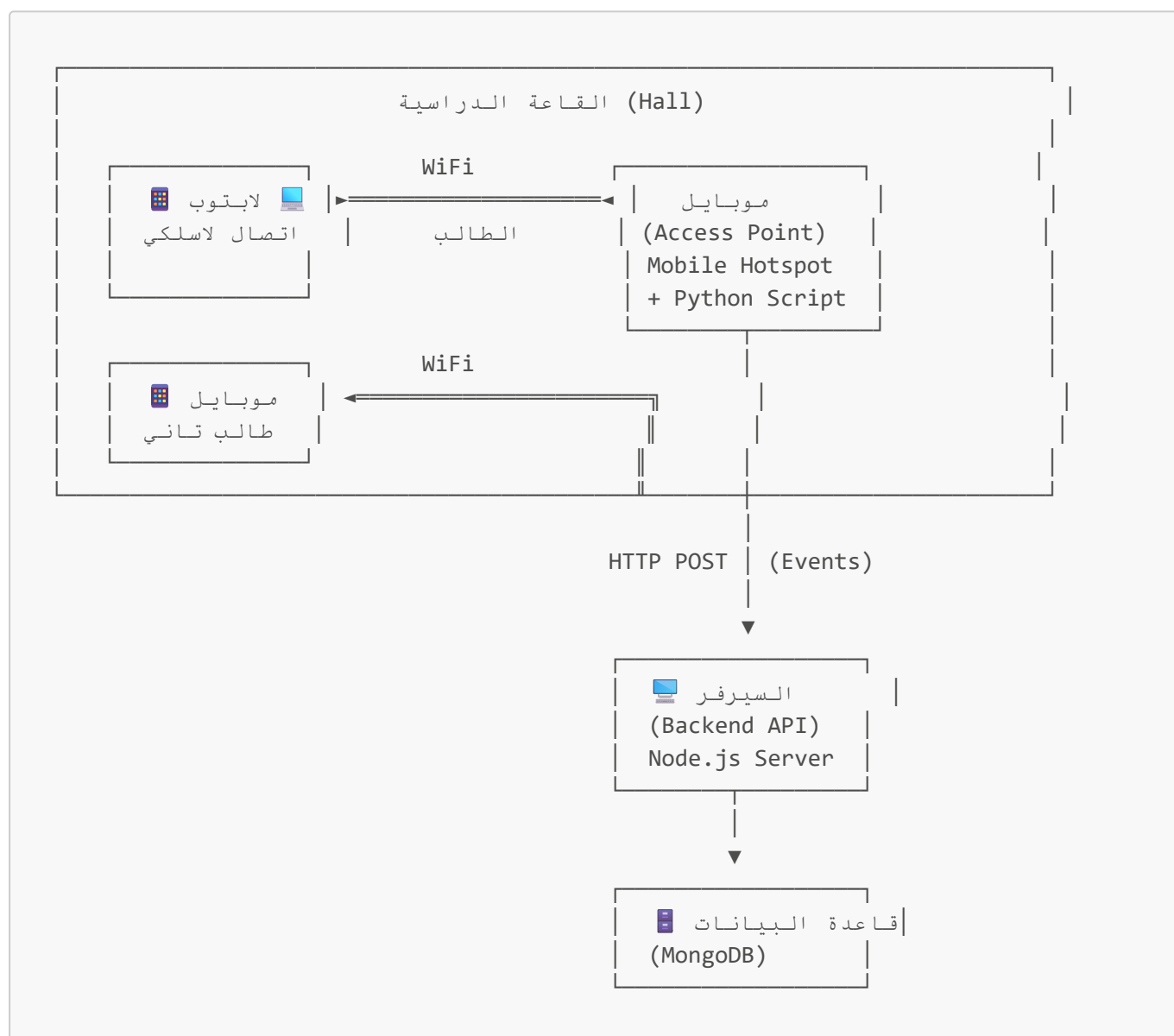
نظرة عامة على الفكرة

تخيل إن عندك قاعة محاضرات في الكلية وعاوز تسجل حضور الطلاب تلقائياً من غير ما حد يعمل أي حاجة يدوي. الحل هو: متصلة بالنظام (Access Point) إننا نستخدم نقطة وصول لاسلكية

- الطالب يدخل القاعة ← موبايله يتصل بالشبكة ← النظام يسجل حضوره تلقائي
- الطالب يسيب القاعة ← موبايله يقطع ← النظام يسجل وقت خروجه

وعليه سكريبت **Mobile Hotspot** عادي - ده لايتوب بيشغل Router هنا مش Access Point ال [!IMPORTANT] عليه سكريبت **Mobile Hotspot** عادي - ده لايتوب بيشغل Router هنا مش Access Point ال [!IMPORTANT] بيقرب مين متصل ومين قطع، وبيبعث المعلومات دي للسيرفر.

المعمارية العامة (System Architecture)



مكونات النظام

1. (السكربت على اللابتوب) Access Point الـ

في القاعة Hotspot بيشتغل على اللابتوب اللي بيعمل `access_point.py` اسمه Python ده ملف

2. السيرفر (Backend API)

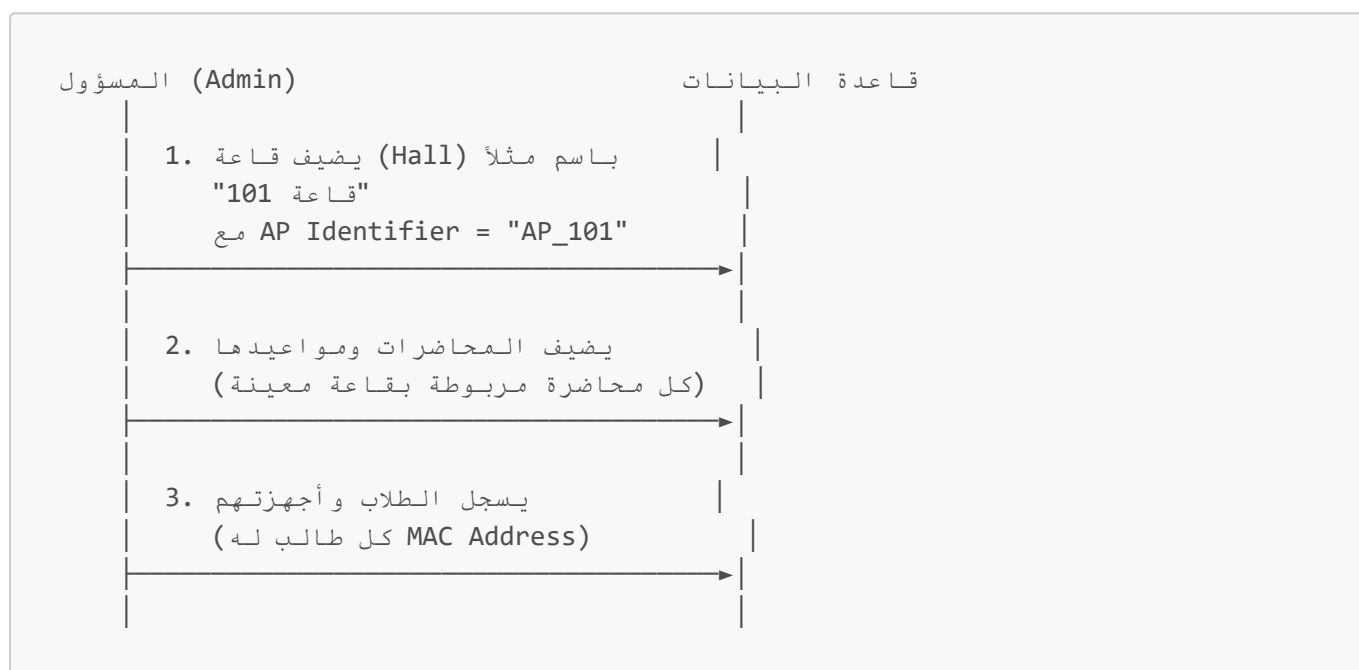
ويعالجها Access Point من الـ (Events) بيستقبل الأحداث Node.js Server ده

3. قاعدة البيانات (MongoDB)

بتخزن بيانات الطلاب والقاعات والمحاضرات وسجلات الحضور

🔄 دورة حياة النظام كاملة (Full System Lifecycle)

(Setup) المرحلة 1: الإعداد والتشغيل



التجهيز على اللابتوب:

1. (Windows) على اللابتوب **Mobile Hotspot** المسؤول يفتح
2. `x`. بتشتغل على نطاق `192.168.137` Windows شبكة الهوتسبوت الافتراضية في
3. `python access_point.py` يشغل السكربت

الإعدادات الأساسية في السكربت:

الشرح	القيمة	الإعداد
المعرّف الفريد للقاعة - لازم يكون نفس القيمة المسجلة في قاعدة البيانات	"AP_101"	AP_IDENTIFIER
Windows Mobile Hotspot الافتراضي لـ IP نطاق الـ	"192.168.137."	SUBNET
المتوازية في عملية الفحص Threads عدد الـ (Scan)	50	THREAD_COUNT

الشرح	القيمة	الإعداد
عدد مرات الفحص الفاشلة قبل ما يعتبر الجهاز قطع الاتصال	30	MAX_MISSED_PINGS
الفترة بين كل عملية فحص والتانية	ثواني 4	SCAN_INTERVAL
عنوان السيرفر	"http://localhost:5000/api"	BACKEND_URL
مفتاح التوثيق عشان السيرفر يتأكد إن الطلب جاي حقيقي Access Point من	مفتاح سري	AP_API_KEY
كل 60 ثانية ببيعت إشارة "أنا شغال" للسيرفر	ثانية 60	HEARTBEAT_INTERVAL

(Network Scanning) المرحلة 2: فحص الشبكة

بتحصل كل 4 ثواني ويتم بالخطوات دي Access Point دي أهم عملية في الـ

(Ping Sweep) الخطوة 1: مسح الشبكة

في النطاق 192.168.137.2 → 192.168.137.254 IP لكل Ping السكربت بيعمل (محتمل IP عنوان 253)

```

Ping Sweep Process

Thread 1: ping 192.168.137.2 → Reply? ✓ أو ✗
Thread 2: ping 192.168.137.3 → Reply? ✓ أو ✗
Thread 3: ping 192.168.137.4 → Reply? ✓ أو ✗
...
Thread 50: ping 192.168.137.51 → Reply? ✓ أو ✗

(في نفس الوقت للسرعة Thread بيشغل 50)
(بعد ما الـ 50 يخلصوا، يبدأ الـ 50 اللي بعدهم)

```

بيشتغل؟ Ping إزاي الـ

- IP لكل عنوان **ICMP Echo Request** ببيعت
- **ICMP Echo Reply** (يعني "Reply from") لو الجهاز موجود ومتصل بالشبكة → بيرد بـ
- (بعد 500 ميلي ثانية Timeout) لو مفيش جهاز ← ما بيردش
- `ping -n 1 -w 500 <IP>` بيستخدم الأمر
 - `-n 1` واحد بس إرسال Ping
 - `-w 500` الانتظار 500 ميلي ثانية كحد أقصى

Multi-threading؟ بيستخدم

- **دقيقتين** لكل فحص = ms واحد ورا الثاني = 253 × 500 IP لكل Ping لو عمل
- في نفس الوقت = **بيخلص في ثواني قليلة** Thread باستخدام 50

MAC Address (ARP Lookup): الخطوة 2: استخراج الـ

بتاع كل MAC Address اللي موجودة على الشبكة، بيدور على الـ IP بعد ما يعرف الـ واحد

ARP Table Lookup

الأمر : `arp -a`

النتيجة :

Internet Address	Physical Address	Type
192.168.137.5	A4-B1-C2-D3-E4-F5	dynamic
192.168.137.12	B8-27-EB-AA-BB-CC	dynamic

IP لكل MAC السكربت بيقرأ الجدول ده ويطلع الـ
ويحول له لصيغة موحدة : `A4:B1:C2:D3:E4:F5`

ARP Table: شرح الـ

- **MAC Address** والـ **IP Address** هو بروتوكول بيربط بين الـ ARP (Address Resolution Protocol) الـ
- ARP Table كل جهاز متصل بالشبكة بيكون موجود في الـ
- ARP Table بيضيفه تلقائي في الـ Operating System لجهاز، الـ Ping لما نعمل
- من كل سطر MAC ويستخرج الـ `arp -a` بالأمر ARP Table السكربت بيقرأ الـ

مهم؟ MAC Address ليه الـ

- (مختلف IP بيديله DHCP) **بيتغير** كل مرة الجهاز يتصل **IP Address** الـ
- **ثابت** لكل جهاز ← ده اللي بنعرف بيه الطالب **MAC Address** الـ

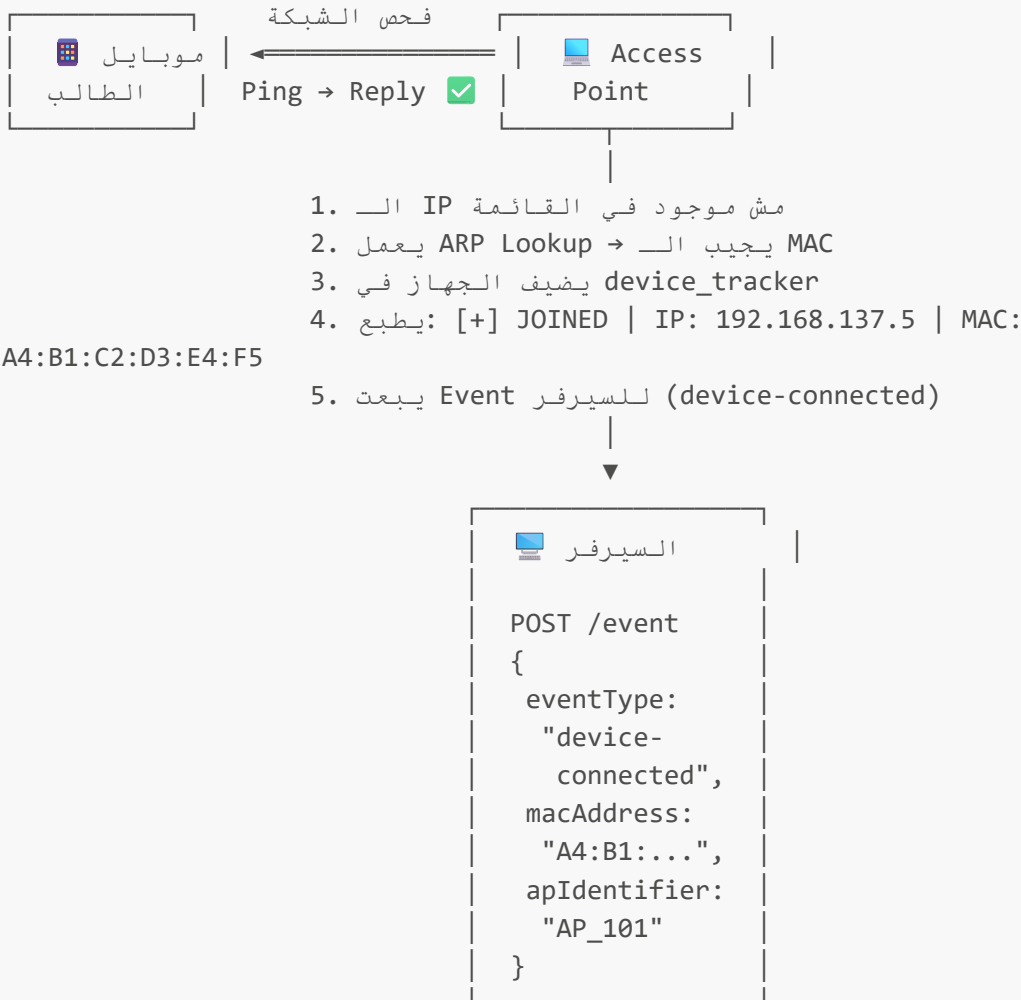
(Device Tracking) المرحلة 3: تتبع الأجهزة

السكربت بيحتفظ بقائمة في الذاكرة فيها كل الأجهزة المتصلة حالياً

```
device_tracker = {
  "192.168.137.5": {
    "mac": "A4:B1:C2:D3:E4:F5",      ← بتاع الموبايل MAC Address الـ
    "missed": 0,                    ← عدد مرات ما لقيناك الجهاز
    "check_in": "2024-03-15 09:05", ← وقت أول اتصال
    "sent_to_server": True          ← هل بلغنا السيرفر؟
  },
  "192.168.137.12": {
    "mac": "B8:27:EB:AA:BB:CC",
    "missed": 3,
    "check_in": "2024-03-15 09:10",
    "sent_to_server": True
  }
}
```

```
}
}
```

● (New Device Connected) السيناريو 1: جهاز جديد اتصل



● (Still Connected) السيناريو 2: جهاز لسه متصل

Ping الجهاز اللي كان موجود لسه بيرد على الـ

↓

يرجعه 0 → "missed" لعداد الـ Reset السكريبت بيعمل

↓

(sent_to_server = False) لو الإرسال للسيرفر كان فشل قبل كده

↓

(Retry Mechanism) بيحاول يبعث ثاني

● (Device Disconnected) السيناريو 3: جهاز قطع الاتصال

آلية اكتشاف قطع الاتصال

Scan 1: 192.168.137.5 → مش موجود missed = 1
 Scan 2: 192.168.137.5 → مش موجود missed = 2
 Scan 3: 192.168.137.5 → مش موجود missed = 3
 ...
 Scan 30: 192.168.137.5 → مش موجود missed = 30

⚠ missed >= MAX_MISSED_PINGS (30)

↓

- ✓ للسيرفر "device-disconnected" الجهاز فعلاً قطع ← يبعث
- ✓ يشيل الجهاز من القائمة
- ✓ (Duration) يحسب مدة التواجد

ليه بنستنى 30 مرة فحص (وليس مرة واحدة)؟

مرة فحص × 4 ثواني بين كل فحص = دقيقتين تقريباً قبل ما نقرر إن الطالب مشي فعلاً. ده عشان ممكن 30 [!TIP] يحصل:

- واحد بس بيرجع بعدها Ping فقدان مؤقت للإشارة ← الموبايل مرد على
- الإشارة ضعيفة لكن لسه متصل ← Hotspot الطالب بعيد شوية عن الـ
- ضاع لكن الاتصال لسه قائم Ping ازدحام على الشبكة ← الـ

كثير False Disconnections لو سجلنا القطع من أول مرة، هيكون عندنا

(Server-Side Processing) المرحلة 4: معالجة الأحداث في السيرفر

بتحصل سلسلة عمليات، Access Point من الـ (Event) لما السيرفر يستقبل حدث

(Authentication) أولاً: التحقق من الهوية

Middleware: verifyAccessPoint

1. اسمه Header فيه Request السيرفر بيشوف هل الـ X-API-Key

- ✗ (401 Unauthorized) لو مفيش → يرفض الطلب
- ✓ لو موجود → يكمل

2. بالمفتاح السري المخزن عنده API Key يقارن الـ

- ✗ (401 Unauthorized) لو مش متطابق → يرفض الطلب
- ✓ في قاعدة (Hall) لو متطابق → بيدور على القاعة apIdentifier البيانات باستخدام الـ

3. AP: بيحذ حالة الـ

```
- isOnline = true
- lastSeen = الوقت الحالي
```

شغال AP تعرف إن الـ (Dashboard) ده بيخلي لوحة التحكم

(Machine-to-Machine communication) هو جهاز Access Point ؟ لأن الـ **Username/Password** وليس **API Key** ليه بنستخدم [!NOTE] بيكون أبسط وأنسب في الحالة دي API Key مش مستخدم بشري. الـ (Machine communication)

(Hall Identification) ثانياً: تحديد القاعة

السيرفر بيدور على القاعة بالترتيب ده

1. بالـ apIdentifier (مثلاً "AP_101")
 - ↓
 - عن Halls بيبحث في جدول:
 - accessPoint.apIdentifier == "AP_101"
 - ↓
 - ✓ "لقاها؟" ← "قاعة 101"
2. نفسه AP بتاع الـ MAC Address لو ما لقاش → بيدور بالـ
3. حطها Middleware لو ما لقاش → بيستخدم القاعة اللي الـ
 - ↳ "Access Point not registered": لو برضو ما لقاش ← خطأ 404

(Connection Log) ثالثاً: تسجيل الحدث

ConnectionLog بغض النظر عن النتيجة، كل حدث بيتسجل في جدول

Connection Log	
macAddress:	"A4:B1:C2:D3:E4:F5"
hall:	"قاعة 101"
eventType:	"device-connected"
timestamp:	"2024-03-15 09:05:30"
processed:	false → true
processingResult:	"نتيجة المعالجة"

كل الأحداث بتتسجل حتى لو الطالب مش معروف. ده مفيد عشان [!TIP]

- تتبع الأخطاء ← لو طالب اشتكى إنه كان حاضر
- إعادة المعالجة ← الأدمن يقدر يعيد معالجة الحدث بعد ما يسجل الـ MAC

(Student Identification) رابعاً: تحديد هوية الطالب

وبيدور على الطالب صاحبه MAC Address السيرفر بياخد الـ

MAC Address: "A4:B1:C2:D3:E4:F5"



البحث بالترتيب :

1.  الجهاز متحقق منه + MAC Address (isVerified = true)
↓
لقاءه؟ ← الطالب أحمد محمد
2. بدون MAC لو ما لقاش ← يدور بالـ شرط التحقق (للأجهزة اللي الأدمن سجلها يدويًا)
3. Device ID لو ما لقاش ← يدور بالـ (خطة بديلة)
4. ← لو ما لقاش خالص
"الحدث اتسجل بس مفيش طالب مربوط"

بتاعه؟ MAC Address إزاي الطالب بيسجل الـ

- الطالب بيسجل من خلال تطبيق الموبايل
- من الموبايل تلقائي Device ID أو MAC Address التطبيق بياخد الـ
- بيتبع للسيرفر ويتخزن مع بيانات الطالب

(Attendance Recording) المرحلة 5: تسجيل الحضور

دي المرحلة الأهم - لما نعرف مين الطالب وفي أنهي قاعة، بنسجله حضور

(device-connected) عند الاتصال:

خطوات التنفيذ - handleConnect

1. (Student Session) إنشاء/تحديث جلسة الطالب

Student Session

```
student:      "أحمد محمد"
currentHall:  "قاعة 101"
macAddress:   "A4:B1:C2:D3:E4:F5"
connectedAt:  "09:05:30"
isActive:     true
```

2. البحث عن محاضرة نشطة في القاعة

هل فيه محاضرة دلوقت في القاعة؟

السيرفر بيشوف:

- (dayOfWeek = 6) اليوم = السبت
- الوقت الحالي = 09:05
- فيه محاضرة من 09:00 لـ 10:30؟

نعم ← محاضرة "هندسة الشبكات" ☒

لا ← مفيش محاضرة (الحدث يتسجل بس) ☒

3. التحقق من التسجيل في المادة

هل الطالب "أحمد" مسجل في مادة
هندسة الشبكات؟

نعم ← نسجله حضور ☒

لا ← الحدث يتسجل بس مش حضور ☒

4. إنشاء سجل الحضور (Attendance Record)

Attendance Record

```
student: "أحمد محمد"
course: "هندسة الشبكات"
lecture: "المحاضرة 5"
hall: "قاعة 101"
date: "2024-03-15"
status: "in-progress" ⌚
sessions:
  - checkIn: "09:05:30"
    checkOut: null (لسه في القاعة)
```

5. ربط الجلسة بسجل الحضور

Session ← → AttendanceRecord

(device-disconnected) عند قطع الاتصال:

خطوات التنفيذ - handleDisconnect

1. البحث عن جلسة نشطة للطالب في القاعة دي

↓

2. تحديث آخر جلسة حضور

```

sessions:
- checkIn: "09:05:30"
  checkOut: "10:25:00" ← تم التسجيل
  duration: 80 دقيقة

```

3. حساب إجمالي وقت التواجد:
totalPresenceTime = 80 دقيقة

4. تحديث حالة الحضور:
status: "in-progress" → "present" ✓

5. إغلاق الجلسة:
isActive: true → false
disconnectedAt: "10:25:00"

المرحلة 6: الجلسات المتعددة (Multiple Sessions)

الطالب ممكن يدخل ويخرج أكثر من مرة خلال المحاضرة الواحدة. النظام بيتعامل مع ده بذكاء [!IMPORTANT]

سيناريو: الطالب أحمد في محاضرة من 9:00 لـ 11:00

09:05 (device-connected) دخل القاعة ←
Session 1: checkIn = 09:05

09:45 (device-disconnected) خرج يشرب مية ←
Session 1: checkIn = 09:05, checkOut = 09:45
duration = 40 دقيقة
status = present

10:00 (device-connected) رجع تاني ←
Session 2: checkIn = 10:00
status = in-progress (اتفتح تاني)

10:50 (device-disconnected) المحاضرة خلصت وخرج ←
Session 2: checkIn = 10:00, checkOut = 10:50
duration = 50 دقيقة

التقرير النهائي:

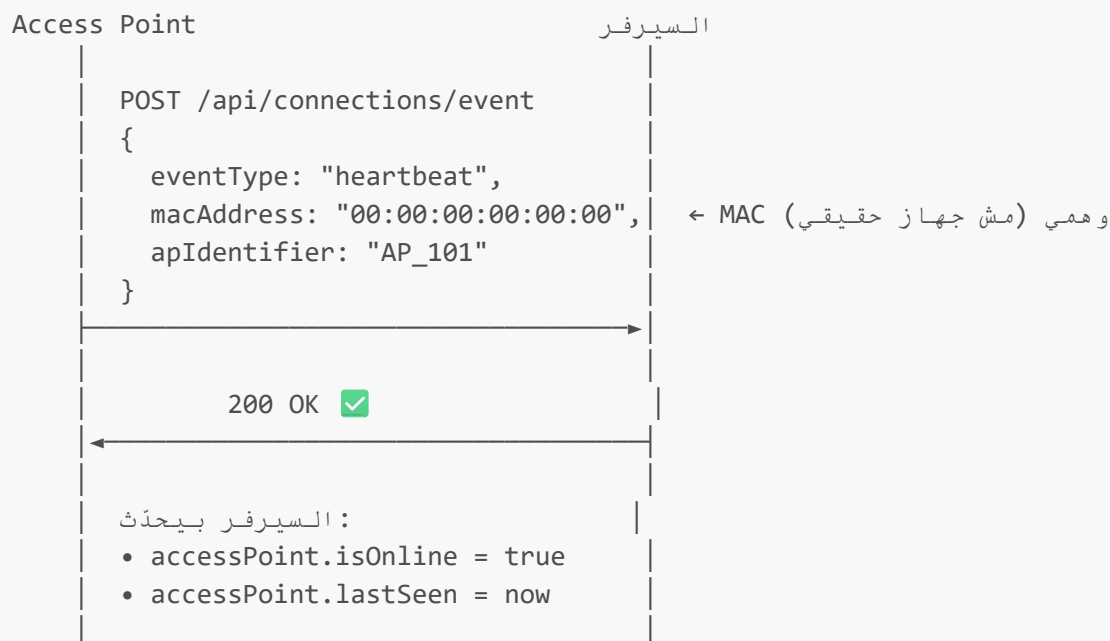
```

إجمالي وقت التواجد = 40 + 50 = 90 دقيقة
مدة المحاضرة = 120 دقيقة
نسبة الحضور = 90/120 = 75%
الحد الأدنى للحضور = 85%
النتيجة = غياب ✗

```

نبضات القلب (Heartbeat) نظام ال

:بيعت إشارة للسيرفر Access Point كل 60 ثانية، الـ



مهم؟ Heartbeat ليه ال

الشرح	الفائدة
شغال ولا لا AP لوحة التحكم بتعرض هل ال	AP معرفة حالة ال
يعني فيه مشكلة، Heartbeat وقف عن إرسال AP لو ال	اكتشاف المشاكل
بيتأكد إن السيرفر شغال Heartbeat عند بداية التشغيل، أول	اختبار الاتصال

ماذا لو السيرفر مش شغال؟

- ييفحص الشبكة عادي ويطبع النتائج ← **Offline** السكربت بيتشغل في وضع
- (Retry) لما السيرفر يرجع يشتغل ← بيحاول بيعت الأحداث المعلقة

🔒 نظام الأمان (Security)

طبقات الأمان

1. 🔑 API Key Authentication
 - لازم يكون معاه مفتاح سري AP كل طلب من الـ
 - X-API-Key اسمه Header المفتاح في
 - لو المفتاح غلط ← الطلب يترفض (401)
2. 📄 AP Registration
 - لازم يكون مسجل في قاعدة البيانات AP الـ

Halls لازم يكون موجود في جدول apIdentifier الـ
لو مش مسجل ← الطلب يترفض (404)

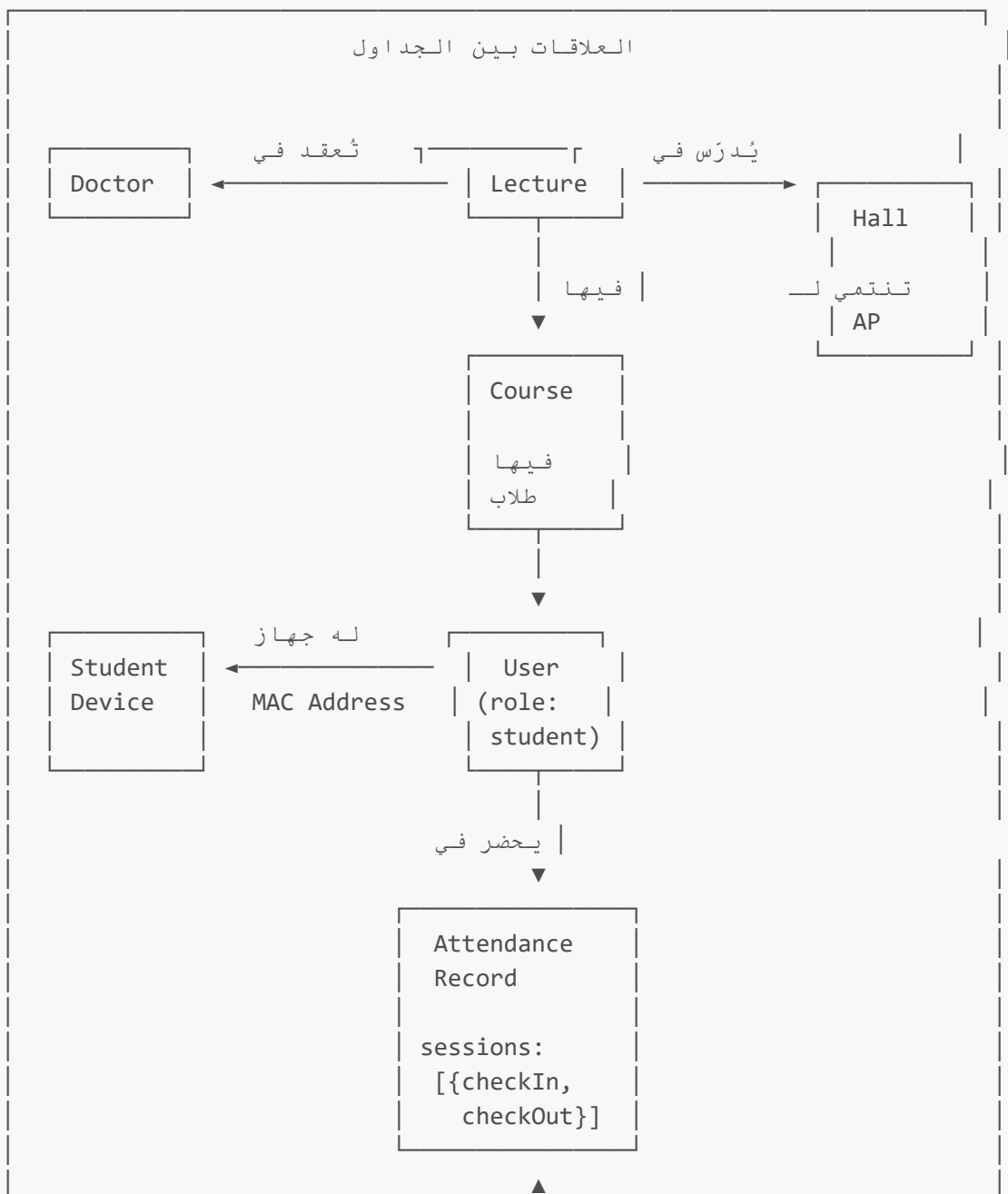
3. 📱 Device Verification

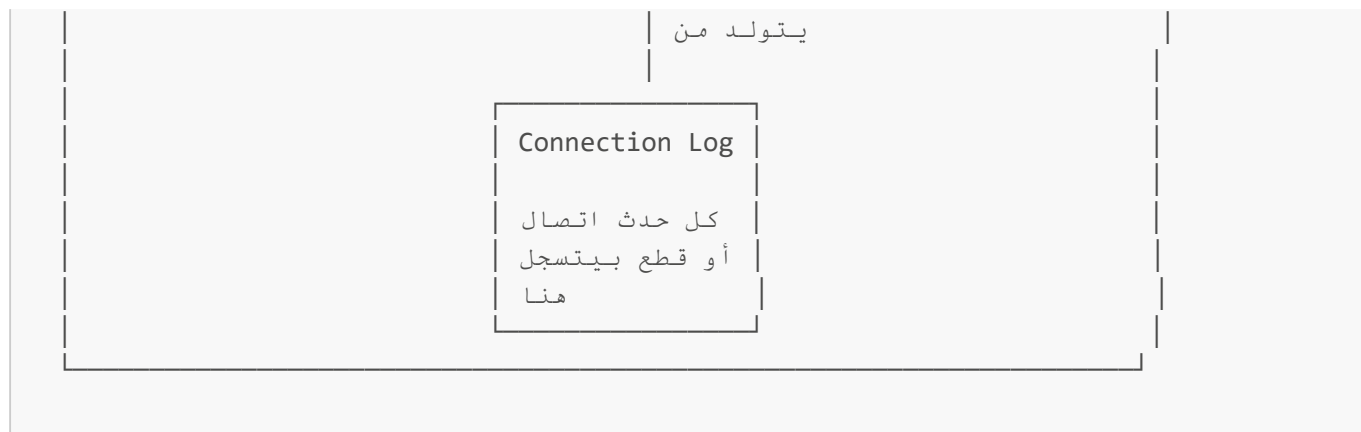
الموبايل لازم يكون مسجل ومتحقق منه
من التطبيق MAC Address الطالب لازم يكون سجل الـ
لو الجهاز مش معروف ← الحدث يتسجل بس مش بيأثر

4. 📖 Course Enrollment Check

حتى لو الطالب معروف، لازم يكون مسجل في المادة
لو مش مسجل ← مش هيتسجله حضور

📊 نموذج البيانات (Data Model)





🌐 (Communication Protocol) بروتوكول الاتصال

للسيرفر Access Point من ال (Request) الطلب:

```
POST /api/connections/event HTTP/1.1
Host: localhost:5000
Content-Type: application/json
X-API-Key: MySuperSecretKey2024
```

```
{
  "eventType": "device-connected",
  "macAddress": "A4:B1:C2:D3:E4:F5",
  "apIdentifier": "AP_101",
  "timestamp": "2024-03-15T09:05:30"
}
```

← نوع الحدث
← MAC الموبايل
← معرف القاعة
← وقت الحدث

من السيرفر (Response) الرد:

```
{
  "success": true,
  "message": "Connection event processed",
  "data": {
    "student": "20201234",
    "hall": "قاعة 101",
    "eventType": "device-connected"
  }
}
```

أنواع الأحداث (Event Types):

الحدث	القيمة	المعنى
اتصال جهاز	device-connected	موبايل طالب اتصل بالشبكة
قطع جهاز	device-disconnected	موبايل طالب قطع الاتصال

المعنى	القيمة	الحديث
"بيقول" أنا شغال ال AP ال	heartbeat	نبضة قلب

🔄 آلية إعادة المحاولة (Retry Mechanism)

ماذا يحصل لو السيرفر مش متاح؟

الفحص الأول:

- └ جهاز جديد اتصل
- └ حاول يبعث للسيرفر ← فشل ❌
- └ sent_to_server = false

الفحص الثاني (بعد 4 ثواني):

- └ الجهاز لسه موجود
- └ يحاول ثاني ← sent_to_server = false
- └ حاول يبعث للسيرفر ← فشل ❌
- └ sent_to_server لسه false

الفحص الثالث:

- └ الجهاز لسه موجود
- └ حاول يبعث للسيرفر ← نجح ✅
- └ مش هيحاول ثاني ← sent_to_server = true

النتيجة: مفيش حدث اتصال بيضيع حتى لو السيرفر وقع مؤقتاً

📊 الصورة الكاملة - من دخول الطالب لحد التقرير

09:00	الدكتور يبدأ المحاضرة (المحاضرة مسجلة في النظام: 09:00 - 10:30)
09:03	Hotspot أحمد يدخل القاعة ← موبايله يتصل بالـ ↓ Ping جديد عن طريق IP يكتشف AP ↓ ARP Table من MAC يجيب الـ AP ↓ للسيرفر MAC + "device-connected" يبعث AP ↓ API Key السيرفر يتحقق من الـ ✅ ↓ "قاعة 101" ← AP_101 يدور على القاعة بالـ ✅ ↓ MAC "أحمد محمد" ← يدور على الطالب بالـ ✅

↓

🖥️

"فيه محاضرة دلوقتي؟" ← "هندسة الشبكات" ✓

↓

🖥️

← أحمد مسجل في المادة؟ ✓

↓

📱

يسجل: checkIn = 09:03 | status = in-progress

10:25

📱

أحمد يخرج من القاعة ← موبايله يقطع

↓

🖥️

فحص فاشل ← أحمد فعلاً مشي 30 ← AP

↓

🖥️

للسيرفر "device-disconnected" + MAC يبعث AP

↓

🖥️

✓ يدور على الجلسة النشطة لأحمد ← لقاه

↓

📱

دقيقة 82: checkOut = 10:25 | يسجل

📱

✓ يحديث: status = present

10:30

📊

لجميع السجلات Finalize المحاضرة تنتهي ← النظام يعمل

التقرير

📊

بعدها:

أحمد محمد

حضور: 82 دقيقة من 90 دقيقة

%نسبة: 91

✓ الحالة: حاضر

⚠️ التعامل مع الحالات الاستثنائية

الحالة	ماذا يحصل
WiFi الموبايل مش متصل بال	النظام مش هيشوف الطالب ← يتسجل غياب
مش مسجل MAC الموبايل متصل بس	بس مش بيأثر على الحضور Log الحدث يتسجل في ال
مفيش محاضرة في الوقت ده	الحدث يتسجل بس مش بيعمل سجل حضور
الطالب مش مسجل في المادة	الحدث يتسجل بس الحضور مش بيتسجل
السيرفر واقع	ويحاول يبعث لما السيرفر يرجع Offline السكريببت يشتغل
(فاشل مؤقت Ping) الإشارة ضعيفة	النظام بيعتني 30 فحص قبل ما يقرر إن الطالب قطع
الطالب خرج ورجع	جديدة وبيتضاف لسجل الحضور Session بيتعمل

🌱 ملخص المكونات التقنية

ACCESS POINT (Python Script)

- Ping Sweep (ICMP) ← اكتشاف الأجهزة
- ARP Table Lookup ← معرفة الـ MAC
- Multi-threading ← سرعة الفحص
- HTTP POST (REST API) ← التواصل مع السيرفر
- Heartbeat ← إثبات إنه شغال
- Retry Mechanism ← ضمان وصول الأحداث

BACKEND SERVER (Node.js)

- API Key Authentication ← AP التحقق من هوية الـ
- Hall Identification ← تحديد القاعة
- Student Identification ← MAC تحديد الطالب بالـ
- Lecture Matching ← ربط الحضور بالمحاضرة
- Session Management ← إدارة جلسات الطلاب
- Attendance Recording ← تسجيل الحضور والغياب

DATABASE (MongoDB)

- Hall + AccessPoint info ← بيانات القاعات
- Lecture Schedule ← جدول المحاضرات
- User + Device ← الطلاب وأجهزتهم
- ConnectionLog ← سجل كل الأحداث
- StudentSession ← الجلسات النشطة
- AttendanceRecord ← سجلات الحضور النهائية

خطوات تشغيل النظام

1.  تأكد إن قاعدة البيانات فيها:
 - apIdentifier = "AP_101" مع (Hall) القاعة
 - المحاضرات ومواعيدها
 - (MAC Addresses) الطلاب وأجهزتهم
2.  على اللابتوب في القاعة:
 - Mobile Hotspot شغل
 - (node server.js) شغل السيرفر
 - (python access_point.py) شغل السكريبت
3.  Hotspot الطلاب يتصلوا بشبكة الـ
4.  النظام يشتغل تلقائي:
 - يفحص الشبكة كل 4 ثواني
 - يسجل الحضور تلقائي
 - يسجل الخروج تلقائي

Hotspot نقطة مهمة للعرض التقديمي: النظام يعتمد على إن الطالب موبايله متصل بشبكة ال [!CAUTION] أو متصل بشبكة ثانية ← مش هيتسجله حضور. لازم يكون الطالب WiFi بتاعة القاعة. لو الطالب موبايله مقفول ال بتاعه في النظام من خلال التطبيق MAC Address مسجل ال